

Interview Assignment - TALENT FINDER RECRUITER PORTAL

You are building a **Recruiter Portal for a recruitment agency/company**. The agency recruiter (user) uses this portal to search and identify suitable candidates for available job positions and submit selected candidates to the corresponding Job Orders.

The portal is designed as a **multi-tenant application**, where different recruitment agencies can use the same portal as separate tenants. Each tenant's Candidates, Job Orders, recruiters, and related data must remain isolated from other tenants.

MODULE INTENT (Anchor for Candidate)

The **Talent Finder Recruiter Portal** provides a foundational, multi-tenant recruitment workspace. A recruiter maintains a pool of **Candidates** (skills, experience, location) and a set of **Job Orders** (required skills, experience, location) within a **Tenant**.

The core value of the portal is **skill matching**: opening a Job Order shows the best-fit candidates from the same tenant, ranked by how many required skills they match, so the recruiter can **Shortlist** them in one click.

The objective of this assessment is to evaluate the candidate's **full-stack capabilities**, "vibe coding" speed, and ability to lay down a solid **architectural/pattern flow** (UI + Backend API + Database) for a multi-tenant application.

Preferred Tech Stack:

- **Frontend UI:** React.js (Tailwind CSS and Redux)
- **Backend API:** Node.js / Express
- **Database:** Any RDBMS (e.g., PostgreSQL, MySQL)

You can also develop using your expertise tech stack as well. Upon completion, please share your developed solution by providing a link to a public GitHub repository.

NOTE: If you encounter any gaps or ambiguities in these requirements, please define your own assumptions and proceed with development. You will have the opportunity to discuss and highlight these assumptions during the interview.

Additionally, feel free to bring your own creativity and innovation to this application! No wireframes or mockups are provided — the screen design, layout, navigation and visual style are entirely yours to imagine. The requirements below describe *what* each screen must let the recruiter do, not how it should look. A thoughtful, original UI is viewed positively.

NOTE ON CV UPLOAD: CV parsing is optional. If you cannot get a PDF/DOCX parsing library working, a working manual "Create Candidate" form is fully acceptable and scores the same on the core requirements. Auto-filling the form from an uploaded CV is a **bonus**, not a requirement.

Technical Deliverables & Evaluation Criteria

Expected Deliverables

1. **Database Schema:** Tables/Collections and demonstrating proper foreign key/reference relationships.
 2. **Backend API:** RESTful or GraphQL endpoints supporting CRUD operations with multi-tenant data isolation, plus skill matching, shortlisting and submissions.
 3. **Frontend UI:** Responsive pages matching the structural requirements outlined below.
 4. **Architectural Flow:** Clean separation of concerns (Routes -> Controllers -> Services -> Repositories).
 5. **Seed Data:** Enough tenants, candidates and job orders that the match ranking is visibly meaningful on first run.
-

Global UI Requirements

To maintain a consistent experience, all modules must adhere to the following layout constraints:

1. **Left Navigation:** A fixed sidebar containing links for **Tenant**, **Candidate**, **Job Order**, and **Submission**.
2. **Top Header:** A clean header section.
 - **Left:** Application Logo/Name.
 - **Right:** User Icon/Avatar.
3. **Tenant Scope:**
 - Tenant wise data should reflect in Candidate, Job Order, Submission view.

Beyond these constraints, the look and feel is open. Choose your own colours, typography, table style, icons and component library.

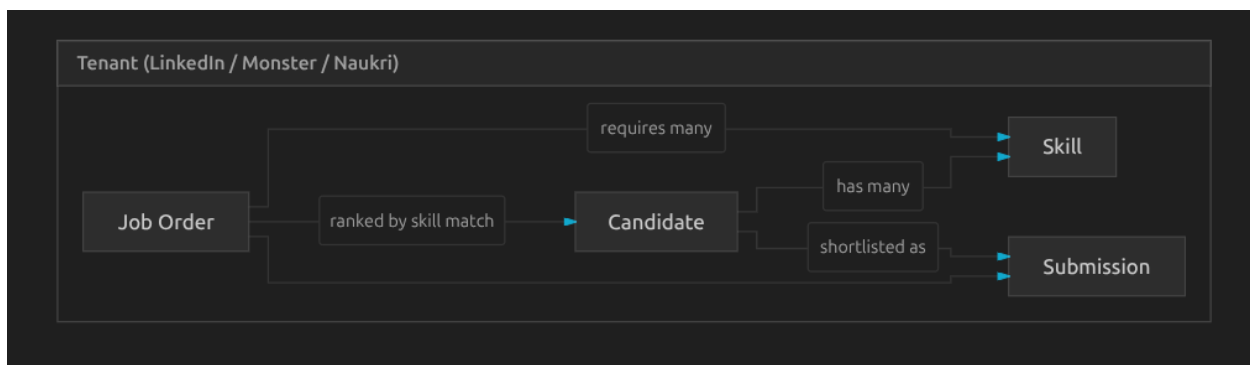
Sample Page Outline (one reference only)

No wireframes are provided. This is a **single sample** of a list page, given only so the page anatomy is clear. Apply a consistent structure of your own to the remaining pages (details, forms) — the styling and arrangement are entirely yours.

+-----	
--	--

- **Page header row** — title on the left; Tenant Dropdown and primary action button on the right.
- **Summary cards** — a couple of at-a-glance counts above the table.
- **Toolbar** — text search on the left, sort control on the right.
- **Table** — the columns named in each user story, plus a row Action Menu (:) with **View**, **Edit**, **Delete**.
- **Footer** — result count on the left, pagination on the right.

Entity Mapping Flow



1. Tenant Module

User Intent

As a system administrator, I want to manage the sourcing channels we recruit through so that candidate and job order data can be segregated appropriately.

User Story 1.1 - Tenant List

As an Admin

I want to see a list of all tenants

So that I can monitor the sourcing channels in use

Acceptance Criteria:

- **Given** tenants exist in the database
- **When** I navigate to the Tenant List page
- **Then** a table is displayed with columns: **Tenant Name**, **Tenant ID**, **Status (Active)**.
- **And** pagination and basic text search are supported.

User Story 1.2 - Create Tenant

As an Admin

I want to create a new tenant

So that a new sourcing channel can be used in the portal

Acceptance Criteria:

- **Given** I click "Create Tenant"
 - **When** I fill out the form with a **Tenant Name** * (required, e.g. **LinkedIn**) and submit
 - **If** valid, the tenant is created and I am redirected to the Tenant List.
-

2. Candidate Module

User Intent

As a recruiter, I want to build a pool of candidates — by uploading their CV or by entering their details manually — so that I can match them to open job orders.

User Story 2.1 - Candidate List

As a Recruiter

I want to view candidates specific to my selected tenant

So that I can browse and manage my talent pool

Acceptance Criteria:

- **Given** I have selected a Tenant from the dropdown
- **Then** the table displays candidates belonging *only* to that tenant.
- **And** columns include **Name**, **Location**, **Experience (Years)**, and **Skills**.
- **And** the list supports Search, Pagination, and Sorting.
- **And** each row has an Action Menu (:) with **View**, **Edit**, and **Delete** options.
- **And** the page header has one buttons: **Add Candidate**.

User Story 2.2 - Create Candidate

As a Recruiter

I want to add a candidate either by uploading their CV or by typing in their details

So that I can register talent quickly, with or without a CV file

Acceptance Criteria:

- **Given** I click "Add Candidate"
- **Then** the form must have the following fields **Full Name**, **Location**, **Total Experience (Years) ***, and **Skills *** (at least one skill).
- **And** the form includes an optional **Upload CV** control at the top, accepting a **.pdf** or **.docx** file.
- **And** the candidate is created against the currently selected tenant.
- **If** valid, the candidate is created and I am taken to the Candidate Details page.

CV Upload - Optional / Good to Have:

- **When** I upload a CV and the file can be read
- **Then** the form fields are **auto-populated** with whatever could be detected: **Full Name**, **Email**, **Phone**, **Location**, **Total Experience** and **Skills**.
- **And** every auto-populated field remains editable — the recruiter confirms the data before saving.
- **If** the CV cannot be read (scanned image, corrupt file, no text), **Then** a friendly message is shown and I simply **type the values into the form fields** instead.
- **If** CV parsing is not implemented at all, **Then** entering the values into the text fields manually is a perfectly acceptable solution and is scored the same on the core requirements.
- *Note: no OCR library is needed for this exercise — see [Appendix: Suggested Free / Open Source Libraries](#).*

User Story 2.3 - Candidate Details & Management

As a Recruiter

I want to view, edit, or delete a candidate

So that I can keep the talent pool accurate

Acceptance Criteria:

- **When** I click "View" from the action menu, **Then** I see the Candidate Details page.
 - **And** the details page shows the profile, the full list of skills, and a link to download the CV when one exists.
 - **And** the details page shows which Job Orders this candidate has been **shortlisted for** along with the submission status, each linking through to the Job Order Details page.
 - **And** the details page has an **Edit** icon and a **Delete** icon.
-

3. Job Order Module

User Intent

As a recruiter, I want to define the roles I am hiring for so that the system can surface matching candidates from my talent pool.

User Story 3.1 - Job Order List

As a Recruiter

I want to see a list of job orders for my selected tenant

So that I can track what I am hiring for

Acceptance Criteria:

- **Given** a tenant is selected
- **Then** the table displays **Job Title**, **Client Name**, **Location**, **Min Experience**, **Number of Openings**, **Required Skills**, and **Status (Open/Closed)**.
- **And** the list supports Search, Pagination, and Sorting.
- **And** the row Action Menu (:) contains **View**, **Edit**, and **Delete**.

User Story 3.2 - Create Job Order

As a Recruiter

I want to create a job order by entering its requirements

So that matching can run against my candidate pool

Acceptance Criteria:

- **Given** I am creating a new job order
- **Then** the form must have the following required fields marked with an asterisk (*): **Job Title**, **Location ***, **Min Experience (Years) ***, **Number of Openings ***, and **Required Skills *** (at least one skill).

- **And** **Number of Openings** * accepts a whole number of 1 or more (how many people are to be hired for this role).
- **And** **Client Name** is optional and **Status** defaults to **Open**.
- **And** there is **no file upload** on this form — job orders are always created by manual entry.
- **If** valid, the job order is created and I am taken to the Job Order Details page.

User Story 3.3 - Job Order Details & Matching Candidates

As a Recruiter

I want to see candidates ranked by how many of the required skills they match

So that I can quickly identify the strongest fits

Acceptance Criteria:

- **When** I view Job Order Details, **Then** I see the job order information and its required skills.
- **And** below it, a **Matching Candidates** list is displayed.
- **And** matching is based on **skill keyword matching**, and candidates are **ranked by the number of skills matched** (highest first).
- **And** each row shows the candidate, the matched skills, and the match count.
- **And** candidates are matched *only* within the same tenant as the job order.
- **And** candidates with no matching skills are not listed.
- **And** each row has a **Shortlist** button.

User Story 3.4 - Shortlist a Candidate

As a Recruiter

I want to shortlist a matching candidate with one click

So that I can track who is progressing for this job order

Acceptance Criteria:

- **Given** I am viewing the matching candidates on a Job Order Details page
- **When** I click the **Shortlist** button for a candidate
- **Then** the candidate is shortlisted against that job order and the button reflects the **Shortlisted** state.
- **And** the candidate appears in a **Shortlisted Candidates** section on the same page.
- **And** a **submission record** is created for that candidate against that job order, visible in the Submission List.

User Story 3.5 - Submission List

As a Recruiter

I want to see all the candidates I have shortlisted across my job orders in one list

So that I can track who has been submitted where, and at what stage they are

Acceptance Criteria:

- **Given** I have shortlisted a candidate from a Job Order Details page
- **When** I open the **Submission** page from the left navigation
- **Then** that candidate appears in the list.
- **And** the table displays **Job Order**, **Candidate**, and **Status**.
- **And** the **Status** of a newly created submission is **Shortlisted**.
- **And** the list supports Search, Pagination, and Sorting.

4. Any AI Feature (Not Mandatory - Good to Have)

Free options are available if you have no API budget (Gemini or Groq free tiers, OpenRouter free models, or a local model via Ollama). Keep API keys in `.env` and never commit them.

Appendix — Suggested Free / Open Source Libraries

Do we need an OCR library to parse PDF/DOC files?

No. CVs exported from Word, Google Docs, LaTeX, Canva, LinkedIn or any job portal carry a real **text layer** inside the file, which free pure-JavaScript libraries read directly — no OCR, no native compilation, no external binaries. OCR is only needed for **scanned or photographed** CVs (an image inside a PDF wrapper), which is a rare edge case here and entirely optional.

A simple rule of thumb: extract the text and, if the result is nearly empty, treat the file as unreadable and open the empty manual form instead.

All packages below are **free and open source**, and all are **MIT licensed** unless noted.

PDF Text Extraction

Package	License	Notes
<code>pdf-parse</code>	MIT	Easiest starting point — one call returns the full text and page count.
<code>unpdf</code>	MIT	Modern, actively maintained, zero native dependencies; works on Node, Bun, Deno and serverless.
<code>pdfjs-dist</code>	Apache-2.0	Mozilla's PDF.js itself — use only if you need text positions/coordinates.

pdf2json	Apache-2.0	Returns structured JSON with positions, useful for two-column CV layouts.
pdf-lib	MIT	Not a text extractor — it creates and edits PDFs. Listed so it isn't chosen by mistake.

DOCX / DOC Text Extraction

Package	License	Notes
mammoth	BSD-2-Clause	The standard choice for .docx; also converts to HTML. Does not read headers/footers.
officeparser	MIT	One API for .pdf, .docx, .xlsx, .pptx and .odt — best value if you want a single dependency.
word-extractor	MIT	Handles legacy .doc as well as .docx, and does read headers/footers.

OCR — Only for Scanned CVs (Optional)

Package	License	Notes
tesseract.js	Apache-2.0	Pure WASM OCR — nothing to install, but slow (seconds per page).
pdf-to-img	MIT	Renders PDF pages to images in pure JS; pair it with tesseract.js.
node-tesseract-ocr	MIT	Faster, but requires the Tesseract binary installed on the machine.

Supporting Packages

Purpose	Package	License
File upload handling (Express)	multer	MIT
Detect real file type from bytes	file-type	MIT
Fuzzy skill matching	fuse.js / string-similarity	Apache-2.0 / MIT
Request & schema validation	zod	MIT
Lightweight NLP / tokenizing	compromise / natural	MIT